Grammar-Derived Operation Abstraction for Cross-Project Vulnerability Detection

Quang-Vinh Dang[a]

[a] *British University Vietnam, Hung Yen, Vietnam*

Abstract

Learned vulnerability detectors generalise poorly across projects, and the field has largely responded with new architectures. We ask instead how much of the gap is attributable to the node representation, and find that the answer is both larger than the architecture effect and simpler than expected. Holding graph topology, backbone, depth, readout and training budget fixed, and varying only what a node carries, we compare project-specific lexical tokens against two project-invariant alternatives: a hand-designed taxonomy of operation types built from API allow-lists and identifier rules, and the parser's own grammar node kinds, which are free. Under project-level GroupKFold with deduplication, mega-projects held train-only, and cluster-bootstrap intervals over held-out projects, both invariant representations beat lexical tokens in every one of eight comparisons across two corpora and three message-passing schemes (+0.049 to +0.106 AUC, seven of eight intervals excluding zero), while the same architectures under a fixed representation differ by roughly a third as much. The engineered taxonomy is not, however, what produces this: on its own it is indistinguishable from lexical features ($p \approx 0.9$), and adding it to grammar kinds gives no consistent gain. We explain this rather than merely reporting it: an information-theoretic analysis shows the taxonomy is 98.2% predictable from the grammar kind alone ($H(\phi \mid \kappa) = 0.06$ bits), so it supplies almost nothing the parser has not already provided, with its residual contribution confined to callee names and operator tokens. We also validate the taxonomy as

Email address: vinh.dq4@buv.edu.vn (Quang-Vinh Dang)

a static analysis against annotated ground truth (0.976 call-site-weighted accuracy), which exposed three defects invisible to downstream accuracy. The practical implication is a one-line change — substitute grammar kinds for lexical tokens — and a methodological one: an engineered abstraction should be checked for redundancy against the free alternative before it is credited with an effect.

Keywords: Program Representation, Static Analysis, Abstract Syntax Trees, Software Vulnerability Detection, Federated Learning, Graph Neural Networks, Differential Privacy

---

1. Introduction

Automated vulnerability detection has become a proving ground for graph learning: a function is parsed into a graph, a network is trained over it, and detection accuracy is reported. Progress is usually claimed architecturally — a different message-passing scheme, an attention variant, a language-model hybrid. Yet replication studies keep reaching the same conclusion: whatever the architecture, performance collapses on projects the model has not seen [1, 2, 3].

This paper examines a factor that most of that work holds constant: what a node in the graph actually *carries*. Nearly all graph-based detectors embed the node's lexical content — an identifier, an API name, a literal. That vocabulary is a property of a codebase, not of a language. FFmpeg allocates with `av_malloc`, GLib with `g_malloc`, the Linux kernel with `kmalloc`. A model whose features come from one project's vocabulary faces largely out-of-vocabulary input on another. The mechanism it must learn — an allocation whose size is computed, an indexed write, a missing bound — is shared; the tokens denoting it are not.

We therefore compare node representations under strict control, holding graph topology, backbone, depth, hidden width, readout, optimiser and gradient budget fixed and varying only the node features. Two project-invariant alternatives to lexical tokens are considered:

- an **engineered operation taxonomy** ϕ, mapping nodes onto eight op-

eration types via API allow-lists, identifier-component rules and syntax-directed cases — the kind of abstraction such work typically constructs; and

- the **parser's own grammar node kinds** κ, an alphabet fixed by the C grammar that requires no design effort whatsoever.

The result was not the one we set out to demonstrate. Abandoning lexical features helps substantially and consistently — every one of eight comparisons across two corpora and three message-passing schemes is positive, with effects of +0.049 to +0.106 AUC and seven of eight intervals excluding zero — and the three architectures differ by roughly a third as much under a fixed representation as the representations differ under a fixed architecture. But the engineered taxonomy is not what produces this. On its own it is statistically indistinguishable from lexical features ($p \approx 0.9$); the grammar kinds carry the effect, and adding the taxonomy to them yields no consistent gain.

We then explain the null result rather than merely reporting it. Measuring the information the taxonomy carries relative to the grammar alphabet shows $H(\phi \mid \kappa) = 0.06$ bits: ϕ is predictable from the node kind alone for 98.2% of nodes, because most of its rules are syntax-directed and therefore already determined by the parser. Its residual contribution is confined to two places — which callee a call expression names, and which operator a binary expression uses. An abstraction that is 96% redundant with a free alternative cannot be expected to add much, and does not.

Separately, we evaluate ϕ *as a static analysis*, annotating 212 call sites and reporting per-class precision and recall. Abstractions of this kind are routinely introduced as preprocessing and never validated. That exercise was not ceremonial: it found that substring-based recognition of wrapped allocators labels `__put_user` and `skb_put`, both writes, as release operations; that classifying `printf` and `fprintf` as buffer-writing formatters counts 655 stream-I/O sites in one corpus as memory operations; and that a permissive copy rule captures SIMD lane extraction. All three are invisible to downstream accuracy.

Our contributions are:

- **A controlled comparison of node representations**, with every other factor held fixed, across two corpora and three message-passing schemes (Sections 4.3 and 4.5).
- **The finding that grammar node kinds suffice**, outperforming both lexical features and an engineered semantic taxonomy, at zero design cost.
- **An information-theoretic explanation** of why the engineered taxonomy adds nothing, and a diagnostic other work can reuse before crediting an abstraction with an effect (Section 4.4).
- **Validation of the abstraction as an analysis**, with per-class precision and recall and the defects it uncovered (Section 4.2).

We report the negative result at the same length as the positive one, because the engineered taxonomy is what we built first and expected to work, and because the redundancy diagnostic that explains its failure is cheap enough that any similar paper could run it.

2. Background and Related Work

2.1. Code Representations for Learned Detectors

Learned vulnerability detection has moved through three representations. Early sequence models treated a function as a token stream: VulDeePecker's code gadgets [4] and SySeVR's syntax- and semantics-based candidates [5] sliced programs into data-dependent token sequences for recurrent classifiers. The Code Property Graph (CPG) [6], unifying abstract syntax, control flow and data dependence in one structure, then made graph learning natural, and Devign [7] established the template still in use: parse to a graph, embed node text, propagate, pool, classify. Later work varied the graph or the propagation — inter-procedural slices in DeepWukong [8], feature-attention over program dependence in IVDetect [9], heterogeneous attention in [10]. A third line embeds

code with pre-trained language models, either alone as in LineVul [11] or fused with a graph channel [12, 13].

What is largely invariant across this progression is the node feature: some embedding of the node's lexical content. Our work isolates that choice. We hold the graph, the propagation scheme and the training protocol fixed and vary only what a node carries, which — to our knowledge — prior work does not do as a controlled comparison.

2.2. Normalisation and Abstraction of Code

Abstracting away identifiers is long-established practice, but as preprocessing rather than as an object of study. Clone detection normalises identifiers and literals so that Type-2 clones become textually identical [6]; sequence-based detectors routinely map user-defined names to placeholder tokens (`VAR1`, `FUN1`) before training [4, 5]; the Devign corpus itself ships a normalised variant of every function.

Two properties distinguish what we do. First, existing normalisation is *anonymising*: it erases the identifier and replaces it with a fresh symbol, preserving only equality of occurrences. Ours is *semantic*: it maps a call onto the memory effect the callee is documented to have, so that `av_malloc` and `kmalloc` become the same symbol as `malloc` rather than three distinct placeholders. Second, and more importantly for a languages venue, we treat the mapping as an analysis with a specification and a measured error rate, rather than as an unexamined preprocessing step. We are not aware of prior work in this area that reports precision and recall for its own abstraction.

The idea of typing operations by their effect is of course not new in programming languages — it is the intuition behind effect systems and behavioural types — but those require type information and whole-program context that function-level vulnerability corpora do not provide. Our ϕ is deliberately weaker: syntax-directed, name-based, and total, so it applies to isolated, non-preprocessed functions, at the cost of the precision quantified in Section 4.2.

2.3. The Cross-Project Generalisation Gap

The finding that motivates this paper is now well replicated. Chakraborty et al. [1] showed that detectors reporting strong within-project results degrade sharply on realistic, project-disjoint data; DiverseVul [14] demonstrated that simply adding training projects does not close the gap; PrimeVul [3] showed that much apparent progress reflects duplication and label noise, with F1 falling dramatically under a rigorous split; and [2] found the same degradation at repository scale.

Responses have been mostly architectural or adaptational — domain adaptation across projects [15, 16, 17], transfer with metric fusion [18]. Our result is complementary and, we argue, prior: with architecture held constant, changing the node representation alone yields a larger and more consistent improvement than the architecture comparisons in Section 4.5. This suggests the gap is at least partly a representation problem that architectural work has been indirectly compensating for.

2.4. Evaluation Methodology in this Area

The methodological pitfalls we control for are documented in exactly the literature above, which is why we treat them as requirements rather than refinements. Duplication across and within corpora inflates results unless removed before splitting [3, 14]. Project mass is heavily concentrated, so a nominally cross-project split can degenerate into evaluation on a single repository unless large projects are handled explicitly. Threshold-dependent metrics are unstable when the calibration and test distributions differ, which they do by construction in cross-project evaluation. And functions within a repository are not independent, so confidence intervals computed over functions understate uncertainty. Section 4.1.4 states how each is addressed.

2.5. Federated and Privacy-Preserving Variants

A body of work applies federated learning to software engineering tasks, including defect prediction [19], code clone detection and defect prediction

jointly [20], and vulnerability detection [21, 22], typically with token or metric features; FedProto [23] introduced sharing class prototypes rather than parameters, and differential privacy for learned models follows DP-SGD [24] with Rényi accounting [25]. Because our abstraction removes project-specific vocabulary from every statistic computed over the graph, it composes naturally with such settings, and we report that as a corollary in Section 5. We do not claim a federated contribution: in our own experiments ordinary parameter averaging, without privacy constraints, remained the stronger aggregation strategy.

3. Operation Abstraction

The object of study is a labelling function that replaces a program's project-specific lexical surface with a small alphabet of operation types fixed by the language grammar. This section specifies it, states the projection lattice relating its three granularities, and defines the graph on which it operates. Section 4 then evaluates it twice: as a static analysis, against annotated ground truth, and as a representation, by its effect on cross-project detection.

3.1. Program Graphs

Each function is parsed with `tree-sitter`, an incremental, error-tolerant parser that recovers a usable tree from non-preprocessed C. That property matters here: vulnerability corpora ship isolated functions without their headers, so a parser requiring a complete translation unit cannot be used. From the resulting tree we build $G = (\mathcal{V}, \mathcal{E})$:

- $\mathcal{V}$: named AST nodes in pre-order, capped at 200;
- $\mathcal{E}_{AST}$: parent–child edges, both directions;
- $\mathcal{E}_{SIB}$: consecutive named siblings, giving source order;
- $\mathcal{E}_{DU}$: a def-use approximation linking successive occurrences of the same identifier leaf.

with $\mathcal{E} = \mathcal{E}_{AST} \cup \mathcal{E}_{SIB} \cup \mathcal{E}_{DU}$, giving on average ∼130 nodes and ∼350 edges per function. We state plainly that this is a syntax graph with a data-dependence approximation, not a Code Property Graph [6]: interprocedural control flow and points-to-precise dependence are absent. Section 5.4 discusses what that bounds.

3.2. The Labelling Function ϕ

Let K be the node-kind alphabet of the tree-sitter C grammar ($|K| = 363$) and $\mathcal{T}_{32}$ the operation alphabet of Table 1. The abstraction is a total function

$$\phi : \mathcal{V} \longrightarrow \mathcal{T}_{32} \cup \{\bot\}, \tag{1}$$

where $\bot$ (`UNKNOWN`) labels nodes carrying no operation semantics. ϕ is deterministic and needs no name resolution, no build system and no whole-program context. It is defined by cases on the node's grammar kind:

1. **Call expressions** are labelled by their callee name through ψ (Section 3.2.1); an unrecognised callee yields `CALL_SITE`. User-defined functions, macros and calls through function pointers therefore all receive a label without resolution.
2. **Kind-determined nodes** follow a fixed table: `subscript_expression` → `ARRAY_INDEX`; `field_expression` → `FIELD_ACCESS`; `cast_expression` → `CAST`; `if`/`switch`/`case` → `BRANCH_*`; loops → `LOOP_*`; `break`/`continue`/`goto`/`return` → `JUMP_*`/`RETURN`; `assignment_expression` and initialising declarators → `ASSIGN`; `update_expression` → `ARITH_ADD`.
3. **Operator-determined nodes**: a `binary_expression` is labelled by its operator token ($+-$ → `ARITH_ADD`; $*/$ → `ARITH_MUL`; `%` → `ARITH_MOD`; bitwise → `BITWISE_OP`; shifts → `SHIFT_OP`; equality → `COMPARISON_EQ`; relational → `COMPARISON_REL`; logical → `LOGICAL_OP`). A `pointer_expression` is `PTR_DEREF` or `ADDR_OF` by its prefix operator; unary `!` is `LOGICAL_OP`.
4. **Otherwise** $\phi(v) = \bot$.

3.2.1. Callee classification ψ

ψ maps a callee name to an operation class in two stages.

Stage 1 (exact). A fixed allow-list of C standard-library names, grouped by the memory effect they carry: allocation, reallocation, release, block copy, block set, string copy, concatenation, buffer-writing formatting, length query, and stream I/O.

Stage 2 (component). Production C rarely calls those directly: FFmpeg wraps allocation as `av_malloc`, GLib as `g_malloc`, the Linux kernel as `kmalloc`, OpenSSL as `OPENSSL_malloc`. Stage 2 splits the callee identifier into components on underscore and camel-case boundaries and fires when a *whole component* matches an operation word.

Whole-component matching, rather than substring matching, is load-bearing. Substring matching — the obvious implementation — is measurably wrong on real code: it labels `__put_user` and `skb_put` (writes) as release operations via "put", `gen_new_label` as allocation via "new", `float64_is_zero` (a predicate) as a block set via "zero", and `reallocate_view` as reallocation, while *missing* `copy_to_user`, a genuine bounded copy. Section 4.2 quantifies the resulting error.

We further separate stream I/O from buffer formatting. `printf` and `fprintf` write to a stream and carry no caller-buffer overflow semantics; only the `s*printf` and `scanf` families fill a caller-supplied buffer. The distinction is not cosmetic: in our Devign sample `fprintf` and `printf` alone account for 655 call sites, every one of which a conflated rule would count as a memory operation.

3.3. The Projection Lattice

Three granularities are defined — $\mathcal{T}_{32}$ (finest), $\mathcal{T}_{16}$, $\mathcal{T}_8$ — related by surjections

$$\pi_{16} : \mathcal{T}_{32} \to \mathcal{T}_{16}, \qquad \pi_8 : \mathcal{T}_{16} \to \mathcal{T}_8, \tag{2}$$

each extended to fix $\bot$. Coarser labellings are compositions, $\phi_{16} = \pi_{16} \circ \phi$ and $\phi_8 = \pi_8 \circ \pi_{16} \circ \phi$, so the granularities are nested *by construction* rather than separately specified.

Table 1: The $|\mathcal{T}_8| = 8$ operation alphabet and the $\mathcal{T}_{32}$ classes each abstracts. $\perp$ (`UNKNOWN`) labels nodes with no operation semantics: declarations, identifiers, literals, punctuation.

$\mathcal{T}_8$	abstracts ($\mathcal{T}_{32}$)	carries
MEMORY_ACCESS	MEMORY_{ALLOC,REALLOC,FREE,COPY,SET}, STRING_{COPY,CONCAT,FORMAT,LENGTH}	buffer lifetime, bounds
ARRAY_INDEX	ARRAY_INDEX	indexed access
PTR_DEREF	PTR_DEREF, ADDR_OF, FIELD_ACCESS, CAST	indirection
CONTROL_BRANCH	BRANCH_{IF,SWITCH}, LOOP_{FOR,WHILE}, JUMP_{BREAK,GOTO}, RETURN	reachability
ARITH_OP	ARITH_{ADD,MUL,MOD}, BITWISE_OP, SHIFT_OP	value computation
COMPARISON	COMPARISON_{EQ,REL}, LOGICAL_OP	guards
CALL_SITE	CALL_SITE, IO_CALL	unresolved or non-memory calls
ASSIGN	ASSIGN	value flow

Proposition 1 (Nesting)**.** *For all $u, v \in \mathcal{V}$, $\phi(u) = \phi(v) \Rightarrow \phi_{16}(u) = \phi_{16}(v) \Rightarrow \phi_8(u) = \phi_8(v)$. Equivalently, the partition of $\mathcal{V}$ induced by ϕ_8 is coarser than that induced by ϕ_{16}, which is coarser than that induced by ϕ.*

Proof. Immediate from Eq. (2): ϕ_{16} and ϕ_8 are compositions of ϕ with total functions, and applying a function to equal arguments yields equal results. The induced partitions are the preimages of a chain of surjections, hence a chain of coarsenings. □

The consequence is that varying granularity varies exactly one quantity — the coarseness of a single labelling — so the granularity experiment in Section 4 is interpretable, which a comparison of three independently designed schemes would not be.

3.4. Node Features

A node is represented as the sum of two embeddings, both over *project-invariant* alphabets:

$$\mathbf{x}_v = E_{\text{op}}\big[\phi_8(v)\big] + E_{\text{kind}}\big[\kappa(v)\big], \tag{3}$$

where $\kappa(v) \in K$ is the grammar kind and $E_{\text{op}}, E_{\text{kind}}$ are learned tables with $|\mathcal{T}_8| + 1 = 9$ and $|K| + 1 = 364$ rows. Both alphabets are fixed by the C grammar and the taxonomy, hence identical across projects and corpora. That invariance is the property we claim explains transfer, and it is what Section 4 tests against a lexical baseline under an otherwise identical protocol.

One caveat, stated precisely. Node *features* contain no identifier, but ϕ *consults* identifier text to classify a callee, so a bounded amount of name-derived information — at most $\log_2(|\mathcal{T}_{32}| + 1) \approx 5$ bits per call node — influences the representation. What does not survive is the identifier itself, its spelling, or any project-specific vocabulary.

4. Evaluation

We evaluate the abstraction twice, and report negative results alongside positive ones:

- **RQ1 (as an analysis).** How accurately does ϕ label operations, per class, and what does component-based recognition of wrapped APIs buy and cost relative to exact allow-listing?
- **RQ2 (as a representation).** With graph, backbone and training budget held fixed, how much does replacing lexical node features with the abstraction change cross-project detection?
- **RQ3 (generality).** Does the effect hold across corpora and across message-passing schemes?
- **RQ4 (what does not work).** Do a learned edge-weighting module, an affinity-weighted federated aggregation, or a finer taxonomy improve on their ablations?

4.1. Setup

4.1.1. Corpora

We use BigVul [26], DiverseVul [14] and PrimeVul [3], loaded from pinned public mirrors. Each is sampled to 8,000 functions. We disclose two properties that are usually left implicit and that materially affect cross-project claims: the sample is taken in corpus order rather than at random, and project mass is highly concentrated — in BigVul, Chrome and `linux` together supply about two thirds of the functions.

Table 2: Corpus properties measured on our samples. Project concentration is reported because it determines whether a nominally cross-project split is meaningful: in BigVul the two largest repositories supply two thirds of the functions, which is why they are held train-only (Section 4.1.4).

Corpus	Functions	Prevalence	Projects	Top-2 share	Nodes/Edges
BigVul	8,000	0.054	134	66.3%	90 / 231
DiverseVul	8,000	0.055	580	27.4%	77 / 195
Devign	8,000	0.473	2	100%	134 / 351

Table 2 records these properties. Devign is included for the analysis validation of Section 4.2 but *excluded from all cross-project claims*: with two repositories, a project-disjoint split degenerates to training on one and testing on the other, and any federated or heterogeneity framing over it would be describing a single codebase.

4.1.2. What the abstraction retains and discards

Table 3 gives the node-type distribution under ϕ_8 on BigVul. Three quarters of AST nodes carry no operation semantics and are labelled $\perp$; they still contribute their grammar kind through Eq. (3), so they are not featureless. The distribution also explains a negative result in advance: the memory-operation class that motivates the taxonomy accounts for 0.43% of nodes, so subdividing it (Section 4.6) cannot move an aggregate metric however semantically justified the distinction is.

4.1.3. Deduplication

Near-duplicate functions are removed *before* any split. Vendored code appears under several project names, so without this step a held-out function can have a twin in training and project-disjointness is defeated silently. Signatures are computed over the abstracted node sequence, which catches renamed copies that text hashing misses; 2–3% of functions are removed.

Table 3: Node-type distribution under ϕ_8 (BigVul). $\perp$ (`UNKNOWN`) nodes carry no operation label but retain their grammar kind.

Type	Share	Type	Share
$\perp$ (`UNKNOWN`)	75.73%	`ASSIGN`	3.38%
`PTR_DEREF`	6.46%	`COMPARISON`	2.64%
`CALL_SITE`	5.31%	`ARITH_OP`	1.59%
`CONTROL_BRANCH`	3.97%	`ARRAY_INDEX`	0.49%
		`MEMORY_ACCESS`	0.43%

4.1.4. Cross-project protocol

Evaluation is repeated project-level GroupKFold: projects are dealt into five folds, no project is split, and each fold is used once as the test set. Two provisions matter. First, projects too large to be placed without dominating a fold — Chrome and `linux` — are held *train-only* and reported as such, so that no fold labelled "cross-project" is in fact an evaluation on one repository. Second, all vocabularies and label bucketings are fitted on training projects only. The resulting folds are stable: test fraction 0.071 on every fold, 39–49 test projects per fold.

We report AUC as the primary metric and AUPRC as the imbalanced-data companion, both threshold-free. F1 is reported only against an explicit floor: the trivial classifier that labels every function vulnerable achieves $F_1 = 0.161$ at the 8.8% prevalence of these test sets, which is above several F1 values reported as results in the literature.

4.1.5. Statistical treatment

Fold-level means are reported with standard deviations, but inference uses a *cluster bootstrap* that resamples held-out projects with replacement (1000 resamples), because functions within a repository share style, APIs and often near-duplicate code and are therefore not independent. We report the paired difference between systems with a 95% percentile interval and a two-sided boot-

Table 4: RQ1: ϕ evaluated as a static analysis on 212 annotated callee names from Devign. Call-site-weighted accuracy is 0.976; macro precision 0.976, macro recall 0.938.

Class	Precision	Recall	F_1	tp/fp/fn
`MEMORY_ALLOC`	0.950	0.905	0.927	19/1/2
`MEMORY_REALLOC`	1.000	1.000	1.000	10/0/0
`MEMORY_FREE`	0.950	1.000	0.974	19/1/0
`MEMORY_COPY`	0.857	0.600	0.706	6/1/4
`MEMORY_SET`	1.000	1.000	1.000	2/0/0
`STRING_COPY`	1.000	1.000	1.000	5/0/0
`STRING_CONCAT`	1.000	1.000	1.000	4/0/0
`STRING_FORMAT`	1.000	1.000	1.000	7/0/0
`STRING_LENGTH`	1.000	1.000	1.000	2/0/0
`IO_CALL`	1.000	0.880	0.936	22/0/3
Macro	0.976	0.938	—	

strap p-value. Where an interval crosses zero we say so and draw no conclusion.

4.2. RQ1: ϕ as a static analysis

We extract every callee name in the Devign corpus, sample 212 names stratified over predicted classes (a precision arm) together with a frequency-weighted sample of unmatched names (a recall arm), and annotate the true class from documented API semantics. The annotation criteria are stated in the released script so that a reader can contest a specific label rather than an opaque score: an allocation label requires that the callee's documented purpose is to obtain, resize or release a heap allocation the *caller* then owns; a copy or set label requires that it writes a caller-supplied destination of caller-specified length; `printf`-family calls that write to a stream are I/O, not buffer formatting.

4.2.1. What the validation changed

The exercise was not confirmatory. It changed the analysis three times, and each change is a result about how such abstractions should be built.

Table 5: RQ1: the copy rule at two operating points, showing the precision/recall trade-off the gold set exposed.

Rule	`MEMORY_COPY`			Macro-P	Weighted acc.
	P	R	F_1		
Permissive (any "copy")	0.450	0.900	0.600	0.935	0.971
Precise (adopted)	0.857	0.600	0.706	0.976	0.976

Substring matching is unsafe. Recognising wrapped allocators by substring — the obvious implementation — labels `__put_user` and `skb_put`, both *writes*, as release operations via "put"; `gen_new_label` as an allocation via "new"; `float64_is_zero`, a predicate, as a block set via "zero"; and `reallocate_view` as a reallocation — while missing `copy_to_user`, a genuine bounded copy. Matching whole identifier components removes all of these.

Stream I/O is not buffer formatting. Placing `printf` and `fprintf` in the formatting class causes 655 call sites in this corpus alone to be counted as memory operations. Only the `s*printf` and `scanf` families write into a caller-supplied buffer.

A precision/recall trade-off, made explicit. Table 5 reports the copy rule at two operating points. A permissive needle matching any "copy" component reaches recall 0.900 but precision 0.450, because it captures SIMD lane extraction (`__msa_copy_u_w`) and pixel-format macros (`COPY16TO9_OR_10`), neither of which writes a caller buffer. Requiring an explicit copy primitive, or "copy" adjacent to a direction word, raises precision to 0.857 at the cost of recall 0.600 — it now misses `AV_COPY32` and `av_dict_copy`. We adopt the precise variant, because a false memory-operation label injects a spurious safety-relevant node into the graph whereas a miss merely leaves the node as a generic call site.

4.2.2. Coverage and residual disagreements

Component matching raises coverage of call sites from 4.7% under exact allow-listing to 10.6%. Ten disagreements remain in the sample, and we name

Table 6: RQ2: node-feature mode under an otherwise identical protocol (BigVul, project-level folds, mean ± std). Differences are paired per fold against the lexical baseline; intervals are cluster bootstrap over held-out projects.

Node features	AUC	Δ AUC (95% CI, p)	AUPRC	Δ AUPRC (95% CI, p)	Folds
Trivial (all-positive)	0.500	—	0.088	—	—
Lexical token	0.587 ± 0.050	—	0.125 ± 0.040	—	10
Operation type ϕ_8	0.635 ± 0.027	−0.006 [−0.054, +0.070] p=0.89	0.142 ± 0.025	−0.003 [−0.034, +0.040] p=0.91	5
Grammar kind κ	**0.692 ± 0.018**	**+0.078** [+0.022, +0.130] p=0.007	**0.223 ± 0.036**	**+0.071** [+0.023, +0.114] p=0.002	5
Operation + kind	0.668 ± 0.027	+0.064 [+0.028, +0.112] p=0.001	0.164 ± 0.034	+0.022 [+0.000, +0.049] p=0.047	9

the informative ones rather than aggregating them away. `bdrv_pread` (36 sites) is a positioned block-device read that no lexical rule recognises as I/O. `g_strdup_printf` both allocates and formats, which a single-label taxonomy cannot express; we label it as allocation, the stronger memory effect. `tcg_temp_free` releases an intermediate-representation temporary in QEMU's own allocator discipline rather than heap memory — a release, but not the kind the taxonomy was designed around. These are limitations of a name-based analysis without type information, and they bound what the abstraction can claim.

4.3. RQ2: which representation transfers

Everything except the node features is held fixed: graph topology, backbone (GGNN), depth, hidden width, mean–max readout, classification head, optimiser, class weighting and gradient budget. Four feature modes are compared: project-specific lexical tokens; the engineered operation taxonomy ϕ_8; the parser's grammar node kinds κ; and their sum, which is the combination an author designing such a system would naturally adopt.

Table 6 reports three findings, and the second is the one we did not expect.

Grammar kinds transfer. Replacing lexical tokens with the parser's node-kind alphabet improves AUC by +0.078 ($p = 0.007$) and AUPRC by +0.071 ($p = 0.002$), winning every fold on both metrics. It is also the most stable representation: fold-to-fold AUC deviation is 0.018 against 0.050 for lexical features, consistent with the account that lexical variance is driven by accidental vocabulary overlap between train and test.

Table 7: RQ2b: information carried by the operation taxonomy relative to the grammar kind, measured over all AST nodes in each corpus.

	BigVul	Devign
$H(\phi)$	1.438 bits	1.520 bits
$H(\kappa)$	4.500 bits	4.430 bits
$H(\phi \mid \kappa)$	**0.059 bits**	**0.071 bits**
$I(\phi;\kappa)/H(\phi)$	95.9%	95.3%
ϕ predictable from κ alone	**98.2%** of nodes	**97.3%** of nodes

The engineered taxonomy does not. ϕ_8 — built from API allow-lists, identifier-component rules and syntax-directed cases, and validated as an analysis in Section 4.2 — is statistically indistinguishable from lexical features on both metrics ($p = 0.89$, $p = 0.91$). This is the representation the abstraction was designed to provide, and on this evidence it provides nothing.

Combining them is worse than the grammar kind alone. The sum reaches AUPRC 0.164 against 0.223 for κ alone. Adding a nine-symbol labelling to a 364-symbol one does not merely fail to help; it costs precision, which is what one would expect if the added channel carries little signal but consumes representational capacity.

Absolute performance remains modest and we state it plainly: AUPRC 0.223 against a base rate of 0.088 is a lift of about 2.5×, and cross-project detection is not solved here.

4.4. RQ2b: why the engineered taxonomy adds nothing

A null result is more useful with a mechanism, so we measure the information ϕ carries relative to κ. Both are functions of the same node, so we can compute their joint distribution exactly over a corpus and ask how much of ϕ is already determined by κ.

Table 7 gives a complete explanation. The taxonomy is predictable from the grammar kind alone for 98.2% of nodes, and 96% of its information is already

Table 8: RQ3: both project-invariant representations against the lexical baseline, across corpora and backbones. Differences are paired per fold; intervals are cluster bootstrap over held-out projects. Bold marks intervals excluding zero.

Corpus	Backbone	Grammar kind κ vs lexical		Operation + kind vs lexical	
		ΔAUC (95% CI)	p	ΔAUC (95% CI)	p
BigVul	GGNN	$+$**0.078** $[+0.022, +0.130]$	**0.007**	$+$**0.064** $[+0.028, +0.112]$	**0.001**
BigVul	GAT	$+0.049$ $[-0.032, +0.108]$	0.217	$+$**0.062** $[+0.014, +0.111]$	**0.012**
BigVul	GIN	$+$**0.106** $[+0.053, +0.166]$	**0.001**	$+$**0.070** $[+0.026, +0.135]$	**0.003**
DiverseVul	GGNN	$+$**0.071** $[+0.024, +0.124]$	**0.001**	$+$**0.083** $[+0.043, +0.124]$	$<$**0.001**

present in κ. This follows from the specification: rules 2 and 3 of Section 3.2 are syntax-directed, so for every node kind they decide, ϕ is a constant function of κ and contributes exactly zero. Only two kinds carry residual uncertainty, and they are precisely the name- and token-dependent cases: `call_expression` (is this callee a memory operation?) and `binary_expression` (is this operator a comparison or arithmetic?).

The lesson generalises beyond this taxonomy. An engineered abstraction over a parse tree is at risk of being a coarsening of the grammar alphabet in disguise; whether it is can be checked in a few lines, before an experiment is run, by computing $H(\text{abstraction} \mid \text{grammar kind})$. We would have saved considerable effort by running that diagnostic first, and we recommend it as a routine step.

4.5. RQ3: generality across corpora and architectures

RQ2 is a single corpus and a single backbone. To ask whether the effect is a property of representations rather than of one experimental cell, we repeat the comparison on a second corpus and under two further message-passing schemes, changing only the named factor.

Table 8 supports one claim strongly and refuses another, and we separate them carefully because the distinction is easy to blur.

What generalises: abandoning lexical features. Every one of the eight comparisons is positive, and seven of eight have intervals excluding zero. Both project-invariant representations beat project-specific tokens on both cor-

pora and under all three propagation schemes, with effects of +0.049 to +0.106 AUC. Against this, the *architecture* differences under a fixed representation are smaller: with lexical features the three backbones span 0.563–0.590 AUC on BigVul, a range of 0.027, roughly a third of the representation effect on the same corpus. This is the paper's central quantitative claim, and it is the one supported by every cell.

What does not generalise: a preference between the two invariant representations. On BigVul/GGNN the grammar kind alone is the stronger of the two, and markedly so on AUPRC (0.223 versus 0.164); on BigVul/GIN it is again stronger; on BigVul/GAT it is weaker and its interval crosses zero; on DiverseVul the two are within each other's intervals. We therefore do *not* claim that grammar kinds beat the combined representation. What the evidence supports is narrower and, we think, more useful: the grammar kind is the component that carries the effect, the operation taxonomy alone does not suffice (Section 4.3, $p = 0.89$), and adding the taxonomy to the grammar kind yields no consistent gain — which Section 4.4 explains by showing the taxonomy is 98.2% predictable from the kind.

A practitioner reading Table 8 should take away that the choice worth making is lexical versus grammar-derived, not which grammar-derived variant; and that the cheaper variant, requiring no taxonomy design, allow-list curation or validation, is not measurably worse.

4.6. RQ4: what does not work

We report three negative results at the same length as the positive one, because each corresponds to a mechanism we implemented and expected to help.

Learned edge weighting. A module that learns a soft importance mask over edges, trained jointly with the classifier and regularised toward sparsity, does not improve on its own ablation; removing it leaves AUC unchanged and slightly improves F1.

Affinity-weighted aggregation. In a federated variant, weighting client

contributions by the cosine similarity of their class prototypes does not improve on uniform averaging; the ablation without it wins on the majority of seeds.

Taxonomy refinement. Increasing the operation alphabet from 8 to 16 or 32 types (Proposition 1 guarantees these are strict refinements of the same labelling) does not produce a corresponding gain. The likely reason is visible in the class distribution: the memory-operation class that motivates the taxonomy fires on under 1% of AST nodes, so subdividing it cannot move an aggregate metric.

5. Discussion

5.1. Why the abstraction transfers

The result is consistent with a simple account. A lexical model's features are drawn from the vocabulary of the projects it trained on; at test time on an unseen repository, much of its input is out-of-vocabulary, and what remains correlates with project identity rather than with the weakness. The abstracted model's alphabet is fixed by the C grammar and the operation taxonomy, so its input distribution is, by construction, the same on every project. The stability we observe supports this: fold-to-fold AUC deviation is roughly half that of the lexical model, which is what one expects if lexical variance is driven by accidental vocabulary overlap between train and test.

This also explains why the effect is larger on ranking (AUC) than at the top of the ranking (AUPRC). The abstraction supplies a consistent notion of *what kind of operation* a node performs, which is enough to order functions by risk; it discards the specific API, which is often what distinguishes a genuinely dangerous call from a benign one of the same type.

5.2. Validating an abstraction is not optional

The most transferable methodological point concerns RQ1. Program abstractions are routinely introduced as preprocessing and never evaluated: a taxonomy is listed, a mapping asserted, and downstream accuracy is reported.

Our experience argues this is unsafe. Two defects in our own mapping — substring matching that classified writes and predicates as memory operations, and a formatter class that absorbed 655 stream-I/O call sites — were invisible to every downstream metric and were caught only by annotating call sites and computing per-class precision. Both would have inflated the very statistic we use to motivate the taxonomy.

The cost is modest: a few hundred annotated names. We would encourage the practice as a default for work that introduces a code abstraction, in the same way that a static analysis is expected to report precision and recall.

5.3. Negative results

We implemented a learned edge-weighting module and an affinity-weighted federated aggregation, and expected both to help. Neither improves on its ablation. We report them because the alternative — keeping components a paper names but its data does not support — is how unreproducible results enter the literature, and because the negative result is itself informative: it locates the effect in the representation rather than in the machinery built around it.

The taxonomy-refinement result has a concrete explanation worth recording. The memory-operation class fires on under 1% of AST nodes, so subdividing it into allocation, release, copy and set cannot shift an aggregate metric however well-motivated the distinction is semantically. A finer taxonomy would need to be paired with an evaluation that weights the nodes it refines.

5.4. Limitations

Graph fidelity. Our graphs are tree-sitter syntax trees with sibling and def-use approximation edges, not Code Property Graphs: interprocedural control flow and points-to-precise dependence are absent. Every system we compare consumes the same graphs, so the comparison is internally fair, but the absolute ceiling is bounded by this choice.

Scale and language. Experiments use 8,000 functions per corpus, compact models, and C only. The abstraction is specified over a tree-sitter grammar and

is therefore instantiable for other languages, but we have not demonstrated that, and a taxonomy validated on C should not be assumed to transfer to, say, Rust's ownership discipline.

Sampling and concentration. Corpus samples are taken in file order rather than at random, and project mass is concentrated (Chrome and `linux` supply about two thirds of BigVul). We hold those projects train-only and report it, but a differently constructed sample could yield different absolute numbers.

Label noise. We inherit the labels of the underlying corpora, whose accuracy is itself contested in the literature that produced them.

Effect size. The AUPRC improvement is small and its interval's lower bound is at zero. The claim we defend is that the representation matters more than the architecture choices it is usually compared against — not that cross-project detection is solved.

5.5. A corollary for federated deployment

Because statistics computed over the abstracted graphs contain no project-specific vocabulary, class-conditioned summaries can be shared between organisations without transmitting code, and perturbed to satisfy differential privacy at low dimensionality. We report this as a corollary rather than a contribution: in our experiments, ordinary parameter averaging without any privacy constraint remained the stronger aggregation strategy, and we found no evidence that the prototype-based alternative beats it absent a formal privacy requirement.

6. Conclusion

We asked how much of the cross-project gap in vulnerability detection is attributable to the node representation rather than the model, and answered it by comparing representations under strict control.

The answer has two parts, and the second is the one we did not anticipate. Replacing project-specific lexical tokens with a project-invariant alphabet improves cross-project ranking substantially — more than the differences between the message-passing schemes we compare, under a fixed representation. The gain holds in every one of eight comparisons spanning two corpora and three message-passing schemes (+0.049 to +0.106 AUC), and is roughly three times the spread between those architectures under a fixed representation. But the ingredient that carries it is the parser's own grammar node kinds, which cost nothing to obtain. The engineered operation taxonomy we built for the purpose — API allow-lists, identifier-component rules, syntax-directed cases, validated as a static analysis at 0.976 accuracy — is on its own statistically indistinguishable from lexical features, and adding it to grammar kinds produces no consistent improvement.

We explain that null result rather than leaving it as an anomaly. The taxonomy is predictable from the grammar kind alone for 98.2% of nodes, and 96% of its information is already present in the kind alphabet, because most of its rules are syntax-directed and therefore decided by the parser before the taxonomy is consulted. Its residual contribution is confined to callee names and operator tokens.

Two recommendations follow, both cheap. When a paper introduces an abstraction over a parse tree, measure $H(\text{abstraction} \mid \text{grammar kind})$ before attributing an effect to it; a few lines of counting would have told us in advance what a substantial experimental campaign eventually showed. And when an abstraction is introduced at all, evaluate it as an analysis: annotating 212 call sites exposed three defects — writes classified as deallocations, stream I/O counted as buffer formatting, SIMD lane extraction counted as copying — that no downstream accuracy metric revealed.

For practitioners the implication is a one-line change: feed the graph neural network the parser's node kinds instead of the source tokens. Absolute performance remains modest — an AUPRC lift of roughly 2.5× over base rate — and cross-project detection remains open. But the cheapest available change to the

representation outperformed both the status quo and the engineered alternative we expected to beat them.

CRediT authorship contribution statement

Quang-Vinh Dang: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing — original draft, Writing — review & editing, Visualization, Project administration.

Declaration of competing interest

The author declares that he has no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data availability

All code, the annotated gold set of call-site labels, and the scripts that regenerate every table and figure in this paper are publicly available at `https://github.com/vinhqdang/vulmorph-fed`. The study uses only publicly available corpora, accessed through the public mirrors identified in Section 4.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author used generative AI-assisted tools in order to improve the language and readability of the manuscript. After using these tools, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

References

[1] S. Chakraborty, R. Krishna, Y. Ding, B. Ray, Deep learning based vulnerability detection: Are we there yet?, IEEE Transactions on Software Engineering 48 (9) (2022) 3280–3296. doi:10.1109/TSE.2021.3087402.

[2] P. Chakraborty, K. K. Arumugam, M. Alfadel, M. Nagappan, S. McIntosh, Revisiting the performance of deep learning-based vulnerability detection on realistic datasets, IEEE Transactions on Software Engineering 50 (8) (2024) 2163–2177, arXiv:2407.03093. doi:10.1109/TSE.2024.3423712.

[3] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, Y. Chen, Vulnerability detection with code language models: How far are we?, in: Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE), IEEE, 2025, pp. 469–481, arXiv:2403.18624. doi:10.1109/ICSE55347.2025.00038.

[4] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, Y. Zhong, VulDeePecker: A deep learning-based system for vulnerability detection, in: Proceedings of the Network and Distributed System Security Symposium (NDSS), Internet Society, 2018. doi:10.14722/ndss.2018.23158.

[5] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, Z. Chen, SySeVR: A framework for using deep learning to detect software vulnerabilities, IEEE Transactions on Dependable and Secure Computing 19 (4) (2022) 2244–2258. doi:10.1109/TDSC.2021.3051427.

[6] F. Yamaguchi, N. Golde, D. Arp, K. Rieck, Modeling and discovering vulnerabilities with code property graphs, in: Proceedings of the IEEE Symposium on Security and Privacy (S&P), IEEE, 2014, pp. 590–604. doi:10.1109/SP.2014.44.

[7] Y. Zhou, S. Liu, J. K. Siow, X. Du, Y. Liu, Devign: Effective vulnerability identification by learning comprehensive program semantics via graph

neural networks, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 32, 2019, arXiv:1909.03496.

[8] X. Cheng, H. Wang, J. Hua, G. Xu, Y. Sui, DeepWukong: Statically detecting software vulnerabilities using deep graph neural networks, ACM Transactions on Software Engineering and Methodology (TOSEM) 30 (3) (2021) 1–33. doi:10.1145/3436877.

[9] Y. Li, S. Wang, T. N. Nguyen, Vulnerability detection with fine-grained interpretations, in: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), ACM, 2021, pp. 292–303. doi:10.1145/3468264.3468597.

[10] Y. Luo, W. Xu, D. Xu, Detecting code vulnerabilities with heterogeneous GNN training, International Journal of Information Security 24, arXiv:2502.16835 (2025). doi:10.1007/s10207-025-01132-x.

[11] M. Fu, C. Tantithamthavorn, LineVul: A transformer-based line-level vulnerability prediction, in: Proceedings of the 19th International Conference on Mining Software Repositories (MSR), ACM, 2022, pp. 608–620. doi:10.1145/3524842.3527927.

[12] D. Hin, A. Kan, H. Chen, M. A. Babar, LineVD: Statement-level vulnerability detection using graph neural networks, in: Proceedings of the 19th International Conference on Mining Software Repositories (MSR), ACM, 2022, pp. 596–607. doi:10.1145/3524842.3527949.

[13] R. Liu, Y. Wang, H. Xu, J. Sun, F. Zhang, P. Li, Z. Guo, Vul-LMGNNs: Fusing language models and online-distilled graph neural networks for code vulnerability detection, Information Fusion 115 (2025) 102748, arXiv:2404.14719. doi:10.1016/j.inffus.2024.102748.

[14] Y. Chen, Z. Ding, L. Alowain, X. Chen, D. Wagner, DiverseVul: A new vulnerable source code dataset for deep learning based vulnerability de-

tection, in: Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses (RAID), ACM, 2023, pp. 654–668. doi:10.1145/3607199.3607242.

[15] S. Liu, G. Lin, L. Qu, J. Zhang, O. D. Vel, P. Montague, Y. Xiang, CD-VulD: Cross-domain vulnerability discovery based on deep domain adaptation, IEEE Transactions on Dependable and Secure Computing 19 (1) (2022) 438–451. doi:10.1109/TDSC.2020.2984505.

[16] C. Zhang, B. Liu, Y. Xin, L. Yao, CPVD: Cross project vulnerability detection based on graph attention network and domain adaptation, IEEE Transactions on Software Engineering 49 (8) (2023) 4152–4168. doi:10.1109/TSE.2023.3285910.

[17] X. Li, Y. Xin, H. Zhu, Y. Yang, Y. Chen, Cross-domain vulnerability detection using graph embedding and domain adaptation, Computers & Security 125 (2023) 103017. doi:10.1016/j.cose.2022.103017.

[18] Z. Cai, Y. Cai, X. Chen, G. Lu, W. Pei, J. Zhao, CSVD-TF: Cross-project software vulnerability detection with TrAdaBoost by fusing expert metrics and semantic metrics, Journal of Systems and Software 213 (2024) 112038. doi:10.1016/j.jss.2024.112038.

[19] H. Yamamoto, D. Wang, G. K. Rajbahadur, M. Kondo, Y. Kamei, N. Ubayashi, Towards privacy preserving cross project defect prediction with federated learning, in: Proceedings of the 30th IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), IEEE, 2023, pp. 485–496. doi:10.1109/SANER56733.2023.00052.

[20] Y. Yang, X. Hu, Z. Gao, J. Chen, C. Ni, X. Xia, D. Lo, Federated learning for software engineering: A case study of code clone detection and defect prediction, IEEE Transactions on Software Engineering 50 (2) (2024) 296–321. doi:10.1109/TSE.2023.3347898.

[21] C. Zhang, T. Yu, B. Liu, Y. Xin, Vulnerability detection based on federated learning, Information and Software Technology 167 (2024) 107371. doi:10.1016/j.infsof.2023.107371.

[22] P. Zhou, M. Hu, X. Xie, Y. Liu, M. Chen, VulFL: An empirical study of vulnerability detection using federated learning, arXiv preprint-ArXiv:2411.16099 (2024).

[23] Y. Tan, G. Long, L. Liu, T. Zhou, Q. Lu, J. Jiang, C. Zhang, FedProto: Federated prototype learning across heterogeneous clients, in: Proceedings of the 36th AAAI Conference on Artificial Intelligence, 2022, pp. 8432–8440. doi:10.1609/aaai.v36i8.20819.

[24] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, L. Zhang, Deep learning with differential privacy, in: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS), ACM, 2016, pp. 308–318. doi:10.1145/2976749.2978318.

[25] I. Mironov, K. Talwar, L. Zhang, Rényi differential privacy of the sampled gaussian mechanism, in: arXiv preprint, 2019, arXiv:1908.10530.

[26] J. Fan, Y. Li, S. Wang, T. N. Nguyen, A C/C++ code vulnerability dataset with code changes and CVE summaries, in: Proceedings of the 17th International Conference on Mining Software Repositories (MSR), ACM, 2020, pp. 508–512. doi:10.1145/3379597.3387501.